

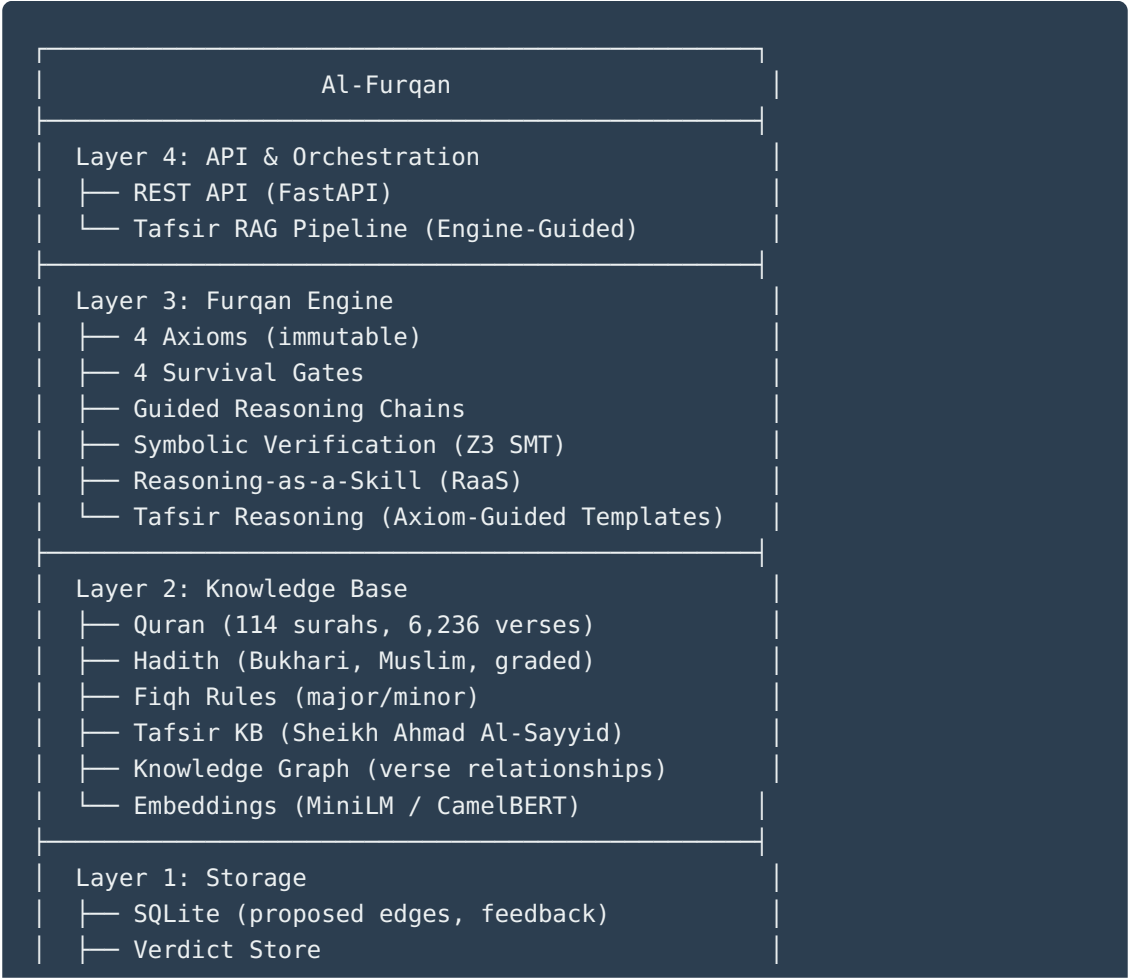
Al-Furqan (الفرقان) — The Criterion

Axiom-Anchored Neuro-Symbolic Reasoning Engine with Tafsir Knowledge Base

Al-Furqan is a **general-purpose reasoning engine** that evaluates ideas, systems, and claims against immutable logical axioms through formal verification. It uses the Islamic Knowledge Base as its verified reference point — because the axioms themselves establish this as the only source that passes all four survival gates.

Key principle: The LLM speaks. The Code thinks. Z3 proves. The Knowledge Base knows.

Architecture



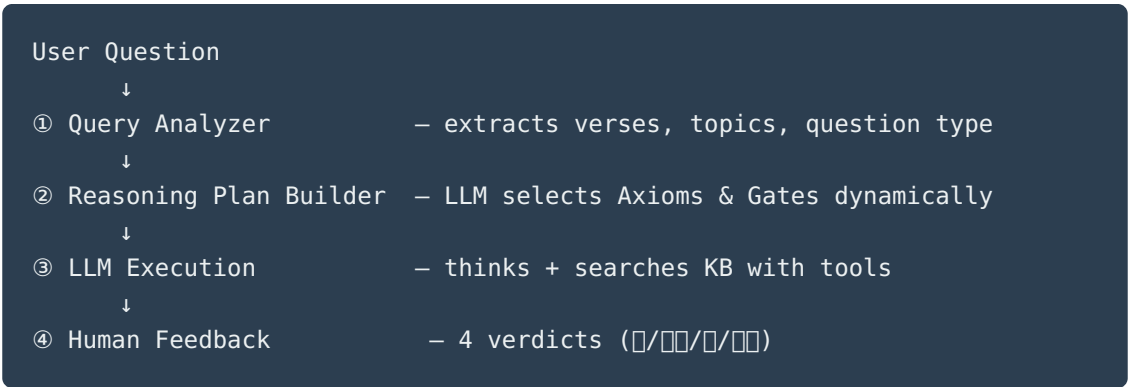
The Four Gates

Every claim passes through 4 sequential survival gates:

Gate	Name	Criterion
1	Source-Integrity	Is raw truth preserved without reduction?
2	Structural-Consistency	Can the system explain causality without luck?
3	Mediation-Zeroing	Is human cognition treated as finite, not authoritative?
4	Origin-Aware	Does truth trace to a transcendent source?

Engine-Guided Tafsir RAG Pipeline

A complete pipeline for Quranic reasoning that teaches the LLM **how to think**, not just what to answer:



KB Tools (exposed to LLM via function calling):

- `search_kb_by_verse("6:5")` — all edges for a verse
- `search_kb_by_topic("السنة الإلهية")` — semantic topic search

- `search_kb_by_relation("6:5", "LINKED_HADITH")` — specific relation type
- `get_verse_context("6:5", range=3)` — surrounding verses + KB data

Reasoning Templates:

6 templates, each mapped to relevant Axioms + Gates: TAFSIR · VERSE_LINK · ISTINBAT · COMPARISON · SEERAH_LINK · GENERAL

Tafsir Knowledge Base:

- **Source:** مدارسة سورة الأنعام — الشيخ أحمد السيد (23 episodes)
- **Extraction:** Whisper transcription → chunk → LLM extraction → human review
- **Current:** 67 entries (Episode 1), central verses: 6:1, 6:5
- **Structure:** Central verse → linked verses, hadiths, concepts, tafsir

Symbolic Verification

Z3 SMT solver provides formal mathematical proofs for: - Transcendence
Necessity Proof - Final Court Necessity Proof - Axiom consistency verification

Test Coverage

705 tests total		
└ Engine (gates, chains, pipeline, axioms, Z3)		~560
└ Knowledge Base (quran, hadith, fiqh, graph)		~60
└ Tafsir RAG Pipeline		100
└ Query Analyzer		23
└ KB Tools		17
└ Tool Executor		13
└ Reasoning Plan Builder		22
└ Pipeline (end-to-end)		13
└ Feedback System		12
└ RaaS (Reasoning-as-a-Skill)		31
└ Memory (MCP server)		56
└ Security (prompt guard, audit, sandbox)		~30

□ Security

5-layer defense-in-depth: 1. **Prompt Guard** — injection detection + sanitization 2. **Axiom Integrity** — SHA-256 hash verification (immutable) 3. **Output Validator** — response filtering 4. **Adapter Sandbox** — isolated LLM execution 5. **Audit Logger** — hashed question logging

□ Project Structure

```
al-furqan/
├── src/al_furqan/
│   ├── engine/                                # Furqan Engine
│   │   ├── axioms.py                         # Immutable axioms + gates
│   │   ├── pipeline.py                       # Scan → Mirror → Verdict
│   │   ├── chains/                          # Guided reasoning chains
│   │   ├── gates/                           # 4 survival gates
│   │   ├── symbolic/                        # Z3 formal verification
│   │   ├── security/                        # 5-layer security
│   │   └── tafsir/                          # Tafsir reasoning pipeline
│   │       ├── axiom_selector.py             # Dynamic axiom/gate selection
│   │       ├── reasoning_plan_builder.py
│   │       ├── reasoning_templates.py
│   │       ├── pipeline.py                   # End-to-end RAG pipeline
│   │       └── feedback.py                   # Human feedback system
│   ├── kb/                                   # Knowledge Base
│   │   ├── collections/                     # Quran, Hadith, Fiqh
│   │   ├── graph/                          # Knowledge graph
│   │   ├── ingestion/                      # Transcript → KB extraction
│   │   └── tafsir/                          # Tafsir KB tools
│   │       ├── query_analyzer.py            # Question analysis
│   │       ├── kb_tools.py                  # 4 search tools for LLM
│   │       └── tool_executor.py             # Tool call execution
│   ├── providers/                           # LLM providers (Ollama, DashScope, Anthropic, etc.)
│   ├── store/                               # Verdict + feedback storage
│   └── api/                                 # REST API
├── data/
│   ├── quran/                              # Complete Quran text
│   ├── hadith/                              # Hadith collections
│   ├── tafsir/                              # 9 Arabic tafsir books (consolidated)
│   ├── lessons/                             # Transcribed episodes
│   ├── review/                              # Proposed edges (KB)
│   ├── benchmark/                           # Evaluation results
│   └── tafsir_feedback/                     # Human feedback entries
├── furqan-raas/                             # Reasoning-as-a-Skill (MCP)
├── furqan-memory/                           # Memory system (MCP)
└── tests/                                   # 705 tests
```

└ scripts/	# Processing & evaluation scripts
└ docs/	# Documentation

Documentation

Document	Description
Architecture v3.0	Complete system architecture
RAG Implementation Plan	Engine-Guided RAG pipeline design
Fine-Tuning Plan	SFT + DPO plan for Furqan-27B
Knowledge Graph Docs	KB schema & extraction
Security Policy	5-layer security architecture
RaaS Docs	Reasoning-as-a-Skill
Sprint Report 2026-03-22	Latest sprint (RAG pipeline)

Roadmap

Done

- [x] Furqan Engine (4 gates, reasoning chains, Z3 verification)
- [x] Knowledge Base (Quran, Hadith, Fiqh, 9 Tafsir books)
- [x] Knowledge Graph (transcript extraction, central verse tracking)
- [x] Security (5-layer defense, axiom integrity)
- [x] Engine-Guided RAG Pipeline (query analysis → reasoning plan → LLM + tools → feedback)
- [x] Human Feedback System (4 verdicts, full context storage)
- [x] Dynamic Axiom/Gate Selection (LLM chooses from raw Engine definitions)
- [x] 705 tests passing

▣ **Next**

- [] Download & process episodes 2-23 (KB expansion)
- [] Collect 500+ human-reviewed pipeline responses
- [] Semantic search (FAISS + embeddings)
- [] API endpoint for the pipeline

▣ **Future**

- [] Fine-tune Furqan-27B (SFT + DPO on Qwen3.5-27B-Claude-Opus-Distilled)
- [] Local deployment (vLLM/SGLang)
- [] Multi-scholar KB support
- [] QLP v3.0 integration (Local-First)

▣ **Tech Stack**

Layer	Technology
Language	Python 3.12
LLM Providers	DashScope (Qwen), Anthropic, Ollama, OpenAI-compatible
Formal Verification	Z3 SMT Solver
Embeddings	MiniLM / CamelBERT
Storage	SQLite (WAL mode)
API	FastAPI
Testing	pytest (705 tests)
Transcription	OpenAI Whisper
Future Training	LLaMA-Factory + Unsloth (LoRA/QLoRA)

□ Team

1. ماجد عارف (Magid Arif) — CEO
 2. محمود السمان (Mahmoud Al-Samman) — Principal Software Architect
 3. محمد الأشماوي (Muhammad Al-Ashmawy) — Research Lead & Domain Expert
 4. آية أبو الوفا (Aya Abu Al-Wafa) — Researcher
 5. مصطفى مرزوق (Mustafa Marzouk) — Researcher
 6. علي (Ali) — Researcher
 7. مريم ياسر (Maryam Yasser) — Researcher
-

□ License

Confidential — Variiance R&D

Al-Furqan: The Code thinks. The LLM speaks. Z3 proves. The Knowledge Base knows.